

Praxisarbeit: Konsolenspiel Labyrinth (C)

Modul 676 – ipso! Bildung

Autor: Bismark Stainett Perez Guzman

Datum: 01.10.2025

1 Management Summary

In dieser Praxisarbeit wurde ein textbasiertes Konsolenspiel in der Programmiersprache C entwickelt. Das Ziel war es, die Grundlagen der Programmiertechnik praktisch anzuwenden. Der Spieler bewegt sich in einem Labyrinth, um einen Schatz zu finden. Das Spiel enthält Hindernisse, die den Weg versperren, und prüft nach jedem Schritt, ob der Schatz erreicht wurde.

Die Umsetzung zeigt, dass auch mit einfachen Mitteln wie einem zweidimensionalen Array und einer klaren Strukturierung ein funktionales Spiel entstehen kann. Durch das Testen konnte die Korrektheit sichergestellt werden. Die Arbeit vermittelt anschaulich, wie Programmsteuerung, Algorithmen und Datenstrukturen in C eingesetzt werden können.

2 Anforderungen und Aufgabenstellung

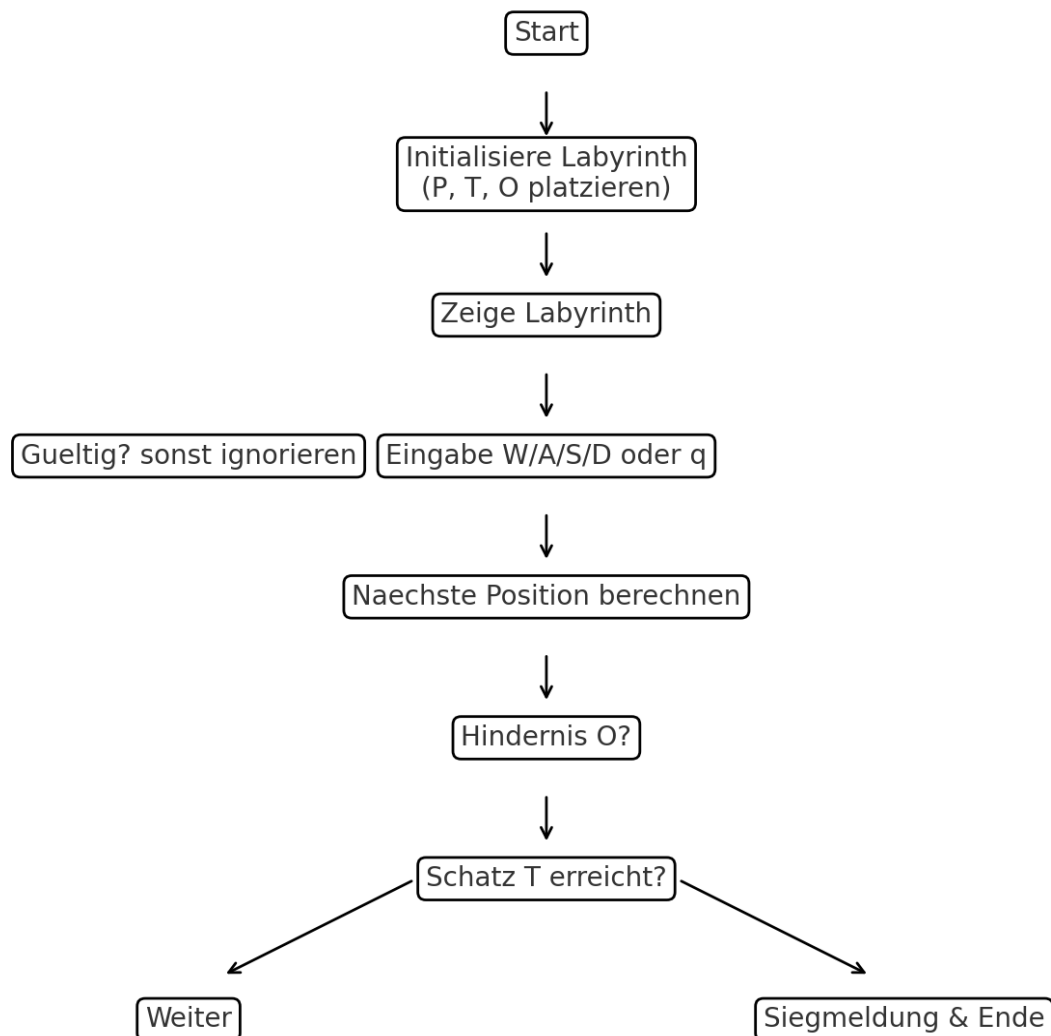
Die Aufgabenstellung bestand darin, ein Konsolenspiel mit folgenden Funktionen zu erstellen:

- Anzeige des Labyrinths
- Bewegung des Spielers über W/A/S/D
- Suche und Finden des Schatzes
- Beenden des Spiels

Die Spiellogik basiert auf einem 2D-Array. Der Spieler (P), der Schatz (T) und Hindernisse (O) werden zufällig platziert. Nach jedem Zug wird die Siegbedingung geprüft. Besonderer Wert lag auf der sauberen Struktur, Eingabeprüfung und einfacher Erweiterbarkeit.

3 Design

Das Spiel wurde vor der Implementierung modelliert. Ein Flussdiagramm beschreibt die einzelnen Schritte: Initialisierung, Eingabe, Bewegung, Siegrprüfung und Ende.



4 Implementierung

Die Implementierung erfolgte in C mit folgenden wesentlichen Bestandteilen:

Datenstrukturen

Es wurden Strukturen für Positionen (Koordinaten) und den Spielzustand verwendet. Dies erleichtert die Verwaltung des Spielfelds.

Initialisierung

Das Labyrinth wird mit Punkten (.) gefüllt, dann werden Spieler, Schatz und Hindernisse zufällig gesetzt. Dabei wird sichergestellt, dass Spieler und Schatz nicht auf demselben Feld starten.

Anzeige

Die Spielfelder werden zeilenweise ausgegeben. Eine Legende erklärt die Symbole.

Eingabe

Spielzüge werden mit W/A/S/D gesteuert. Mit q kann das Spiel beendet werden. Ungültige Eingaben werden ignoriert.

Bewegung und Kollision

Die Bewegung prüft Spielfeldgrenzen und blockiert Hindernisse (O). Nur gültige Züge verändern die Spielerposition.

Siegbedingung

Wenn die Spielerposition die Schatzposition erreicht, wird eine Siegesmeldung ausgegeben und das Spiel beendet.

5 Test

Das Programm wurde mit mehreren Durchläufen getestet. Die Screenshots belegen die Korrektheit der Spiellogik.

```
Labyrinthspiel (W/A/S/D und Enter, q zum Beenden)
```

```
. . . . . . . . .  
. O O . . O O . . .  
. O . O . . . . . O  
. . . O . . . P . .  
. . . . . O . . . .  
. . . . . . . O O .  
. . . . . . . . . O  
O . O . . . . . . .  
. . . T . . . . O .  
. . . . . . . . .
```

```
Legende: P=Spieler  T=Schatz  O=Hindernis  .=frei
```

```
Bewege den Spieler (W/A/S/D): wasd
```

```
. . . . . . . . .  
. O O . . O O . . .  
. O . O . . . P . O  
. . . O . . . . . .  
. . . . . O . . . .  
. . . . . . . O O .  
. . . . . . . . . O  
O . O . . . . . . .  
. . . T . . . . O .  
. . . . . . . . .
```

```

Legende: P=Spieler  T=Schatz  O=Hindernis  .=frei
Bewege den Spieler (W/A/S/D): a

. . . . .
. 0 0 . . 0 0 . . .
. 0 . 0 . . . . 0
. . . 0 . . . . .
. . . . 0 . . . . .
. . . . . 0 0 .
. . . . . . 0
0 . 0 . . . . .
. . . P . . . 0 .
. . . . . . .

Legende: P=Spieler  T=Schatz  O=Hindernis  .=frei

Herzlichen Glueckwunsch! Du hast den Schatz gefunden!
[1] + Done          "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microsoft-MIEng
ine-In-prnejqdz.fdw" 1>"/tmp/Microsoft-MIEngine-Out-4pxgnuws.mgc"
@Stainett →/workspaces/c-kurs-ipso (main) $ 

```

Die Tests zeigen:

- Startzustand mit zufälliger Platzierung von Spieler, Schatz und Hindernissen.
- Bewegungen mit W/A/S/D funktionieren, Züge gegen Hindernisse werden blockiert.
- Beim Erreichen des Schatzes erscheint die Meldung „Herzlichen Glückwunsch! Du hast den Schatz gefunden!“ und das Spiel endet.

6 Lessons learned

Die Arbeit zeigte, wie wichtig eine saubere Strukturierung und Modularisierung auch bei kleinen Projekten ist. Besonders hilfreich war die Trennung in kleine Funktionen für Initialisierung, Ausgabe und Spiellogik. Herausfordernd war die saubere Eingabeprüfung, die aber für die Stabilität entscheidend ist.

Gelernt wurde außerdem, wie mit einfachen Algorithmen und Datenstrukturen ein funktionierendes Spiel entsteht. Für die Zukunft wären Erweiterungen wie mehrere Schätze, Level oder ein Zeitlimit denkbar.

7 Anhang

Kurzanleitung zur Nutzung:

- Kompilieren: `gcc -std=c11 -O2 -g main.c -o labyrinth`
- Starten: `./labyrinth`
- Steuerung: W/A/S/D für Bewegung, q zum Beenden.
- Symbole: P = Spieler, T = Schatz, O = Hindernis, . = freies Feld.

Die Projektziele wurden erreicht und die Aufgabenstellung vollständig erfüllt. Das Spiel ist funktionsfähig, nachvollziehbar dokumentiert und kann erweitert werden.